Alternatively, you can use a different validation framework, such as `FluentValidation`, as you'll see in chapter 32.

Whichever validation approach you use, it's important to remember that these techniques don't protect your application by themselves. The Razor Pages framework automatically executes the validation code after model binding, but it doesn't do anything different if validation fails! In the next section we'll look at how to check the validation result on the server and handle the case where validation has failed.

16.3.2 Validating on the server for safety

Validation of the binding model occurs before the page handler executes, but note that the handler always executes, whether the validation failed or succeeded. It's the responsibility of the page handler to check the result of the validation.

NOTE Validation happens automatically, but handling validation failures is the responsibility of the page handler.

The Razor Pages framework stores the output of the validation attempt in a property on the `PageModel` called `ModelState`. This property is a `ModelStateDictionary` object, which contains a list of all the validation errors that occurred after model binding, as well as some utility properties for working with it.

As an example, listing 16.7 shows the `OnPost` page handler for the Checkout.cshtml Razor Page. The `Input` property is marked for binding and uses the `UserBindingModel` type shown previously in listing 16.6. This page handler doesn't do anything with the data currently, but the pattern of checking `ModelState` early in the method is the key takeaway here.